
NAME: S.DHARSHINI

REG NO: 192424258

SUB CODE: DSA0502

SUB NAME: QUERY PROCESSING FOR DATA SCIENCE

CO1 ASSESSMENT TOOL 1

15. Airline Passenger Analysis

Files:

Flights.csv

Passengers.csv

Flights.json

Flights.xml.

Read passenger datasets from CSV, JSON, and XML. Merge passenger and flight records, calculate occupancy rate, identify delayed flights, summarize airport-wise passenger traffic, and export the processed dataset.

AIM: To analyze airline passenger and flight data by reading datasets from CSV, JSON, and XML formats, merging records, calculating occupancy rate, identifying delayed flights, summarizing airport-wise passenger traffic, and exporting the processed dataset.

FILES: Flights.csv, Passengers.csv ,Flights.json, Flights.xml

PROCEDURE:

- Create the project folder in VS Code.
- Install required Python libraries.
- Import the pandas library.

- Read the CSV, JSON, and XML files.
- Merge passenger and flight datasets using **Flight_ID**.
- Calculate the occupancy rate using occupied and total seats.
- Filter delayed flights based on the flight status.
- Group passenger records by airport to summarize passenger traffic.
- Export the processed dataset to a CSV or Excel file.
- Verify the output.

CODE:

```
import pandas as pd

# Read Passenger CSV
passengers = pd.read_csv("Passengers.csv")
print("Passengers Data")
print(passengers)

# Read Flights CSV
flights = pd.read_csv("Flights.csv")
print("\nFlights Data")
print(flights)

# Read Flights JSON
flights_json = pd.read_json("Flights.json")

print("\nFlights JSON Data")
print(flights_json)

# Read Flights XML
flights_xml = pd.read_xml("Flights.xml")

print("\nFlights XML Data")
print(flights_xml)
```

```
# Merge Passenger and Flight records
merged_data = pd.merge(passengers, flights, on="Flight_ID", how="inner")

print("\nMerged Data")
print(merged_data)

# Calculate Occupancy Rate
merged_data["Occupancy_Rate"] = (
    merged_data["Occupied_Seats"] / merged_data["Total_Seats"]
) * 100

print("\nOccupancy Rate")
print(merged_data[["Flight_ID", "Occupancy_Rate"]])

# Identify Delayed Flights
delayed_flights = merged_data[merged_data["Status"] == "Delayed"]

print("\nDelayed Flights")
print(delayed_flights)

# Airport-wise Passenger Traffic
airport_traffic = merged_data.groupby("Airport")["Passenger_ID"].count()

print("\nAirport-wise Passenger Traffic")
print(airport_traffic)

# Export Processed Dataset
merged_data.to_csv("Processed_Airline_Data.csv", index=False)

print("\nProcessed dataset exported successfully!")
```

OUTPUT:

```
4      P105      Karan      F102      Bangalore      Air India      Bangalore      Delhi      200      180      Delayed

D:\Desktop\AirlinePassengerAnalysis>python airline_analysis.py

Passengers Data
Passenger_ID Passenger_Name Flight_ID Airport
0      P101      Rahul      F101      Chennai
1      P102      Priya      F102      Bangalore
2      P103      Amit      F101      Chennai
3      P104      Sneha      F103      Delhi
4      P105      Karan      F102      Bangalore

Flights Data
Flight_ID  Airline  Source Destination Total_Seats Occupied_Seats Status
0      F101  IndiGo  Chennai  Mumbai  180      160      On Time
1      F102  Air India Bangalore Delhi  200      180      Delayed
2      F103  SpiceJet Delhi  Kolkata  150      120      On Time

Flights JSON Data
Flight_ID  Airline  Source Destination Status
0      F101  IndiGo  Chennai  Mumbai  On Time
1      F102  Air India Bangalore Delhi  Delayed
2      F103  SpiceJet Delhi  Kolkata  On Time

Flights XML Data
Flight_ID  Airline  Source Destination Status
0      F101  IndiGo  Chennai  Mumbai  On Time
1      F102  Air India Bangalore Delhi  Delayed
2      F103  SpiceJet Delhi  Kolkata  On Time

Merged Data
Passenger_ID Passenger_Name Flight_ID  Airport  Airline  Source Destination Total_Seats Occupied_Seats Status
0      P101      Rahul      F101      Chennai  IndiGo  Chennai  Mumbai  180      160      On Time
1      P102      Priya      F102      Bangalore Air India Bangalore Delhi  200      180      Delayed
2      P103      Amit      F101      Chennai  IndiGo  Chennai  Mumbai  180      160      On Time
3      P104      Sneha      F103      Delhi  SpiceJet Delhi  Kolkata  150      120      On Time
4      P105      Karan      F102      Bangalore Air India Bangalore Delhi  200      180      Delayed
```

CONCLUSION:

The airline passenger analysis successfully combines data from multiple file formats, computes occupancy rates, identifies delayed flights, summarizes airport traffic, and exports the processed results for further analysis.

16. Railway Reservation Analysis

Files:

Trains.csv

Tickets.csv

Tickets.json

Tickets.xml

Read railway reservation datasets from multiple formats. Merge train and ticket records, calculate occupancy percentage, identify cancelled tickets, summarize route-

wise passenger count, and export the cleaned dataset.

AIM: To analyze railway reservation data collected from CSV, JSON, and XML files by merging records, calculating occupancy percentage, identifying cancelled tickets, generating route-wise passenger summaries, and exporting a cleaned dataset.

FILES: Trains.csv , Tickets.csv , Tickets.json , Tickets.xml.

PROCEDURE:

- Create a project folder in VS Code.
- Place all dataset files in the project folder.
- Install the Pandas library.
- Import the required Python libraries.
- Read `Trains.csv`.
- Read `Tickets.csv`.
- Read `Tickets.json` and convert it to a DataFrame.
- Read `Tickets.xml` and convert it to a DataFrame.
- Merge all ticket datasets.
- Merge the ticket data with the train data using `Train_ID`.
- Calculate occupancy percentage for each train.
- Identify cancelled tickets.
- Generate a route-wise passenger count.
- Remove duplicate records.
- Handle missing values.
- Export the cleaned dataset to `cleaned_dataset.csv`.

- Display the final results.

CODE:

```
import pandas as pd
import json
import xml.etree.ElementTree as ET

# -----
# STEP 1 : Read Trains.csv
# -----

trains = pd.read_csv("Trains.csv")

print("\n===== TRAINS DATA =====")
print(trains)

# -----
# STEP 2 : Read Tickets.csv
# -----

tickets_csv = pd.read_csv("Tickets.csv")

print("\n===== TICKETS CSV =====")
print(tickets_csv)

# -----
# STEP 3 : Read Tickets.json
# -----

with open("Tickets.json", "r") as file:
    json_data = json.load(file)

tickets_json = pd.DataFrame(json_data)

print("\n===== TICKETS JSON =====")
print(tickets_json)

# -----
```

```

# STEP 4 : Read Tickets.xml
# -----

tree = ET.parse("Tickets.xml")
root = tree.getroot()

xml_list = []

for ticket in root.findall("Ticket"):

    xml_list.append({

        "Ticket_ID": int(ticket.find("Ticket_ID").text),
        "Train_ID": int(ticket.find("Train_ID").text),
        "Passenger_Name": ticket.find("Passenger_Name").text,
        "Status": ticket.find("Status").text

    })

tickets_xml = pd.DataFrame(xml_list)

print("\n===== TICKETS XML =====")
print(tickets_xml)

# -----
# STEP 5 : Merge all Ticket Files
# -----

tickets = pd.concat(
    [tickets_csv, tickets_json, tickets_xml],
    ignore_index=True
)

print("\n===== ALL TICKETS =====")
print(tickets)

# -----
# STEP 6 : Remove Duplicate Records
# -----

```

```

tickets = tickets.drop_duplicates()

# -----
# STEP 7 : Merge Train & Ticket Data
# -----

merged = pd.merge(
    tickets,
    trains,
    on="Train_ID",
    how="left"
)

print("\n===== MERGED DATA =====")
print(merged)

# -----
# STEP 8 : Fill Missing Values
# -----

merged.fillna("Unknown", inplace=True)

# -----
# STEP 9 : Passenger Count
# -----

passenger_count = merged.groupby("Train_ID").size()

trains["Passengers"] = trains["Train_ID"].map(passenger_count)

trains["Passengers"] = trains["Passengers"].fillna(0)

# -----
# STEP 10 : Occupancy Percentage
# -----

trains["Occupancy (%)"] = (
    trains["Passengers"] /
    trains["Capacity"]
) * 100

```



```

print("\n===== OCCUPANCY =====")
print(trains)

# -----
# STEP 11 : Cancelled Tickets
# -----

cancelled = merged[
    merged["Status"] == "Cancelled"
]

print("\n===== CANCELLED TICKETS =====")
print(cancelled)

# -----
# STEP 12 : Route-wise Passenger Count
# -----

route_summary
merged.groupby("Route").size().reset_index(name="Passenger_Count")

print("\n===== ROUTE SUMMARY =====")
print(route_summary)

# -----
# STEP 13 : Export Cleaned Dataset
# -----

merged.to_csv(
    "cleaned_dataset.csv",
    index=False
)

print("\nCleaned dataset exported successfully.")

print("\nFile Name : cleaned_dataset.csv")

print("\n===== PROGRAM COMPLETED =====")

```

OUTPUT:

```
File Edit Selection View Go Run ... Railway_Reservation_Analysis

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

D:\Desktop\Railway_Reservation_Analysis>python railway_analysis.py

===== TRAINS DATA =====
Train_ID Train_Name Route Capacity
0 101 Chennai Express Chennai-Madurai 500
1 102 Pandian Express Madurai-Coimbatore 400
2 103 Rockfort Express Chennai-Trichy 350

===== TICKETS CSV =====
Ticket_ID Train_ID Passenger_Name Status
0 1 101 Ravi Booked
1 2 101 Priya Cancelled
2 3 102 Kumar Booked

===== TICKETS JSON =====
Ticket_ID Train_ID Passenger_Name Status
0 4 101 Anu Booked
1 5 103 Vijay Booked

===== TICKETS XML =====
Ticket_ID Train_ID Passenger_Name Status
0 6 102 Deepa Booked
1 7 103 Arun Cancelled

===== ALL TICKETS =====
Ticket_ID Train_ID Passenger_Name Status
0 1 101 Ravi Booked
1 2 101 Priya Cancelled
2 3 102 Kumar Booked
3 4 101 Anu Booked
4 5 103 Vijay Booked
5 6 102 Deepa Booked
6 7 103 Arun Cancelled

===== MERGED DATA =====
Ticket_ID Train_ID Passenger_Name ... Train_Name Route Capacity
Ln 17, Col 11 Spaces: 4 UTF-8 CRLF () XML 5:33 PM 18-Jul-26
```

```
File Edit Selection View Go Run ... Railway_Reservation_Analysis

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

===== MERGED DATA =====
Ticket_ID Train_ID Passenger_Name ... Train_Name Route Capacity
0 1 101 Ravi ... Chennai Express Chennai-Madurai 500
1 2 101 Priya ... Chennai Express Chennai-Madurai 500
2 3 102 Kumar ... Pandian Express Madurai-Coimbatore 400
3 4 101 Anu ... Chennai Express Chennai-Madurai 500
4 5 103 Vijay ... Rockfort Express Chennai-Trichy 350
5 6 102 Deepa ... Pandian Express Madurai-Coimbatore 400
6 7 103 Arun ... Rockfort Express Chennai-Trichy 350

[7 rows x 7 columns]

===== OCCUPANCY =====
Train_ID Train_Name Route Capacity Passengers Occupancy (%)
0 101 Chennai Express Chennai-Madurai 500 3 0.600000
1 102 Pandian Express Madurai-Coimbatore 400 2 0.500000
2 103 Rockfort Express Chennai-Trichy 350 2 0.571429

===== CANCELLED TICKETS =====
Ticket_ID Train_ID Passenger_Name ... Train_Name Route Capacity
1 2 101 Priya ... Chennai Express Chennai-Madurai 500
6 7 103 Arun ... Rockfort Express Chennai-Trichy 350

[2 rows x 7 columns]

===== ROUTE SUMMARY =====
Route Passenger_Count
0 Chennai-Madurai 3
1 Chennai-Trichy 2
2 Madurai-Coimbatore 2

Cleaned dataset exported successfully.
File Name : cleaned_dataset.csv

===== PROGRAM COMPLETED =====
Ln 17, Col 11 Spaces: 4 UTF-8 CRLF () XML 5:33 PM 18-Jul-26
```

CONCLUSION:

The Railway Reservation Analysis project was successfully implemented using Python in VS Code. Data from multiple file formats was integrated into a single dataset, analyzed, cleaned, and exported. This demonstrates how Python and the Pandas library can efficiently perform data integration, cleaning, and analysis tasks.